

FRA in the constrained-belief world, by worked example

FRA constrained-belief sprint (Phase 1 complete; Phase 2 partial)

16 July 2026

Abstract

This is the example-first companion to the formal note (`fra_cbu_note.tex`). It re-derives the same conclusions about feature-resolved attention (FRA) — which attention edits are gauge, which obey a path calculus — but it does so from the concrete numbers of *our own trained models*, one worked arithmetic example at a time, and only then writes down the general formula each example instantiates. The setting is paper P2’s single-Mess3 one-layer transformer (`code/phase1`); every Phase-1 number here is computed *exactly* by enumerating all $3^{10} = 59049$ length-10 sequences with their exact process probabilities, so there are no error bars unless one is named. Read it with a Python session open: each section is (i) a concrete question, (ii) a worked numerical example from a checkpoint we actually trained, (iii) the general formula it instantiates, and (iv) the mistake you would make if you skipped the theory. Where our own runs missed a prediction, the miss is in the text, not hidden.

Contents

1	The world in 9.4 millinats	3
1.1	The question	3
1.2	The process, and the one object that runs the whole note	3
1.3	The constrained update: what the transformer must compute	3
1.4	Worked example: one sequence, by hand	4
1.5	The anchors: how many nats are on the table	4
2	How much is each lag worth?	4
2.1	The question	4
2.2	Worked example: the exact ladder	5
2.3	The shrinkage surprise, and why the sign flips	5
2.4	The trap: the row the gradient never touched	6
3	The dial: what FRA-QK actually measures	7
3.1	The question	7

3.2	The four score blocks	7
3.3	One softmax row, and why a cut is a tilt	7
3.4	Worked example: dialing the pedestal to zero	7
3.5	A corroborating null: cut every token and nothing should happen	8
4	Cutting what flows: FRA-OV is the real handle	9
4.1	The question	9
4.2	The attribution, and why it over-carries: the warp	9
4.3	The skip/diagonal split (the $k=0$ gauge)	10
4.4	Worked example: predicting the tracking slope ex ante	10
4.5	The null, for free	11
4.6	Collateral is geometrically forced	11
4.7	Quadratic CE, and the missing null	11
4.8	The honest misses	11
5	The dictionary you deserve	12
5.1	The question	12
5.2	Worked example: what TopK learned	12
5.3	The channels are contrasts, not directions	13
5.4	The intervention inherits the code	13
5.5	Purity does not see this	14
6	Two heads, one truth	14
6.1	The question	14
6.2	Why one head cannot: a one-line argument	14
6.3	Worked example: the oscillation lives only in the sum	15
6.4	The invariant converges; the split never stops moving	15
6.5	Rich vs. lazy init decides even the invariant	15
7	Coda: the factored world (Phase 2)	16
8	Field guide: five questions before you trust an FRA reading	17

1 The world in 9.4 millinats

1.1 The question

Everything below is about one transformer trained on one data process. Before we can ask what an attention edit *does*, we have to know how much there was to do in the first place. So: how hard is this task, in nats, and what exactly must the network compute to solve it?

1.2 The process, and the one object that runs the whole note

The data is a single **Mess3** hidden-Markov process (P2 App. B; our `mess3.py`), with 3 hidden states and 3 emitted symbols. Two knobs: a transition scale x and an emission concentration a . We take **Config A**: $x = 0.15$, $a = 0.60$. The marginal hidden transition $T = (1 - 2x)I + x(\mathbf{1}\mathbf{1}^\top - I)$ has stationary distribution $\pi = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ and eigenvalues $\{1, \zeta, \zeta\}$ with

$$\zeta = 1 - 3x = 0.55,$$

the doubly-degenerate subleading eigenvalue — the correlation time of the process. This single scalar sets the “how fast does the past decay” rate that the whole note keeps returning to.

The one object you must hold in your head is the **token displacement**

$$g(z) := \pi T^{|z|} - \pi \in \mathbb{R}^3, \quad T^{|z|} := T^{(z)}/P(z), \quad P(z) = \frac{1}{3},$$

the nudge that observing symbol z applies to the belief over the next hidden state. For Config A these are, exactly (`mess3.py`, and reproduced in the formal note’s check (i) to 1.1×10^{-16}):

$$g(0) = \begin{pmatrix} 0.2667 \\ -0.1333 \\ -0.1333 \end{pmatrix}, \quad g(1) = \begin{pmatrix} -0.1333 \\ 0.2667 \\ -0.1333 \end{pmatrix}, \quad g(2) = \begin{pmatrix} -0.1333 \\ -0.1333 \\ 0.2667 \end{pmatrix}.$$

Three facts about these vectors carry a surprising amount of the story. Each sums to zero ($g(z) \cdot \mathbf{1} = 0$), so all three live in the 2-dimensional **belief plane** $\mathcal{P} = \{v : v \cdot \mathbf{1} = 0\}$. They *also* sum to zero across tokens, $\sum_z g(z) = 0$, so any two of them span $\mathcal{P}$ and the third is minus their sum. And they are symmetric: $\|g(z)\|$ is the same for all z , and no two are orthogonal ($g(z) \cdot g(z') = -\frac{1}{2}\|g(z)\|^2$ for $z \neq z'$). Hold onto “two dimensions, three vectors, summing to zero” — in Section 4 it is what makes selective concept removal *geometrically impossible* in this world.

1.3 The constrained update: what the transformer must compute

P2’s central claim is that this transformer does not run exact Bayesian filtering. It runs a linear-in-log-space surrogate, the **constrained belief update**. In spectral form (formal note Eq. (S0.4); `mess3.py`), the prediction after reading a length- d prefix $z_{1:d}$ is

The constrained update.

$$r_1(d) = \pi + u_d, \quad u_d = \sum_{s=1}^d \zeta^{d-s} g(z_s).$$

Every lag- $(d - s)$ token contributes its displacement $g(z_s)$, *geometrically discounted* by ζ^{d-s} . All the lag structure is in the scalar ζ^{d-s} ; all the token structure is in the plane vector $g(z_s)$. That factorization is the reason FRA has anything clean to say here.

1.4 Worked example: one sequence, by hand

Take the concrete length-10 sequence

$$z_{1:10} = (0, 2, 1, 0, 1, 2, 2, 0, 1, 0).$$

Compute $r_1(2)$, the prediction after the first two tokens, by hand. Only lags 0 and 1 exist, so

$$u_2 = \zeta^1 g(0) + \zeta^0 g(2) = 0.55 \begin{pmatrix} 0.2667 \\ -0.1333 \\ -0.1333 \end{pmatrix} + \begin{pmatrix} -0.1333 \\ -0.1333 \\ 0.2667 \end{pmatrix} = \begin{pmatrix} 0.0133 \\ -0.2067 \\ 0.1933 \end{pmatrix},$$

so $r_1(2) = \pi + u_2 = (0.3467, 0.1267, 0.5267)$. Running the exact recursion in `mess3.py` on this sequence returns $(0.34667, 0.12667, 0.52667)$ — identical (we checked all destinations $d = 1, 2, 3, 10$ to $\leq 5 \times 10^{-16}$). The prediction is a π -anchored walk in the belief plane, one geometrically-discounted $g(z_s)$ kick per observed token. That is the entire “computation”; the transformer’s job is to realize this map with attention.

1.5 The anchors: how many nats are on the table

Now the scoreboard. Averaged over query positions with exact process probabilities (`train_log.json`, `anchors.json`), the cross-entropies are

predictor	CE (nats)	what it is
unigram log 3	1.098612	know nothing but the marginal
trained model (Config A)	1.089462	our checkpoint, exact enumeration
r_1 -ansatz (clipped readout)	1.089562	the constrained update, read out linearly
exact Bayes	1.089203	optimal filtering

The distance from “know nothing” to “optimal” is $1.098612 - 1.089203 = 0.009409$ nats = **9.4** mnat. *That is the entire size of this world.* Every attention edit, every FRA reading, every gauge swing in this note plays out inside a 9.4-millinat budget. The trained model sits $1.089462 - 1.089203 = 0.259$ mnat above Bayes, i.e. it captures $(1.098612 - 1.089462)/0.009409 = 97.3\%$ of the available information. Notice also that the trained model (1.089462) actually *beats* the r_1 -ansatz readout (1.089562): the network’s own MLP readout is a touch better than clipping the constrained update. (Config A2, a second seed, lands the other side of the ansatz; the whole seed spread is ≈ 0.25 mnat.)

What would fool you. If you skip this section and dive straight into “mechanism,” every number in the note looks alarmingly small and you will over-read noise. A 1-millinat cut effect is not weak evidence of a weak mechanism; it is *most of a mechanism* in a 9.4-millinat world. Conversely, a “structure” you find that would cost the loss 10^{-5} nats to destroy is, for this task, indistinguishable from nothing. Section 2 makes the “indistinguishable from nothing” floor precise, and it turns out to swallow most of what FRA-QK will later appear to measure.

2 How much is each lag worth?

2.1 The question

The constrained update sums a contribution from every past token. Before we attribute anything to how attention routes those contributions, we should ask the loss which of them it even cares about.

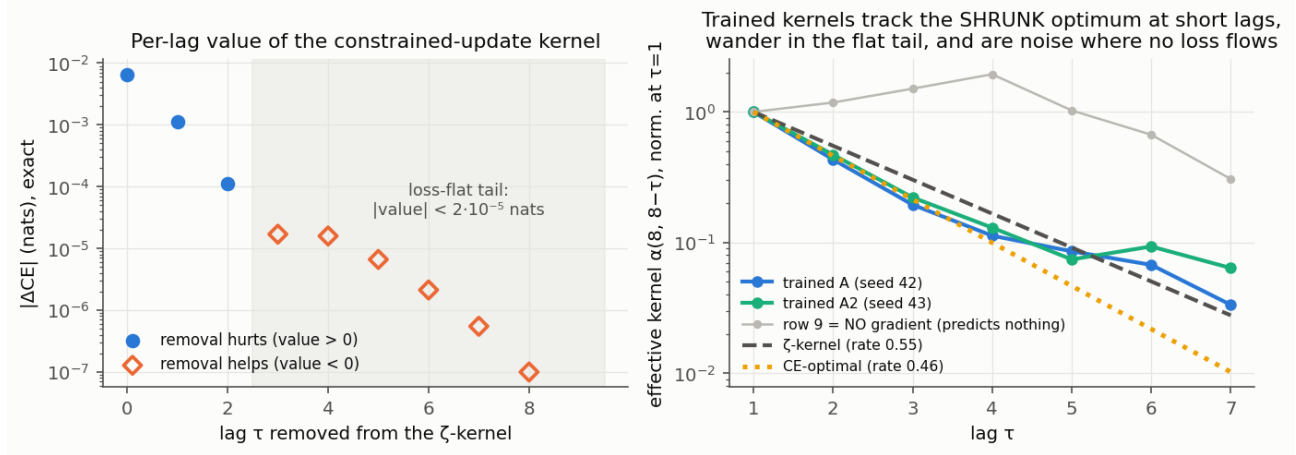


Figure 1: **The value ladder and the trained kernels.** Left: the exact CE value of each lag τ (nats it buys), collapsing to a negative sub- 2×10^{-5} floor beyond $\tau = 2$. Right: trained attention kernels by destination row. Where the loss pins them (lags 1–4) both seeds track the *shrunk* optimum (ratio ≈ 0.43 – 0.45 , below $\zeta = 0.55$); in the flat tail the seeds wander; the gradient-free last row (gray) is an order of magnitude off.

If lag 7 is worth nothing, then whatever the attention pattern does at lag 7 is unconstrained, and a “clean mechanism” found there is a fiction the gradient never signed off on.

2.2 Worked example: the exact ladder

Because Config A is exactly enumerable, we can compute the CE *value of lag* τ : fit the best kernel that uses lags $0..\tau$, and record how many nats the τ -th rung buys over stopping at $\tau - 1$ (`kernel_ce.py`, `out/kernel_ce_A.json`):

lag τ	0	1	2	3	4	5	≥ 6
value (mnat)	6.591	1.106	0.1132	−0.0170	−0.0159	−0.0066	$\leq -2 \times 10^{-6}$

Read the ladder. Lag 0 is worth 6.6 mnat, lag 1 is worth 1.1 mnat, lag 2 is worth 0.11 mnat — and **every lag beyond 2 has a *negative* value below 2×10^{-5} nats**. Negative: including it *hurts*. That sign is not a rounding artifact; it is the tell of double-counting (next subsection). The first three rungs sum to essentially the whole 9.4-millinat budget; the rest of the geometric tail is, to the loss, noise.

2.3 The shrinkage surprise, and why the sign flips

Here is the surprise. The process eigenvalue is $\zeta = 0.55$, so you would guess the CE-optimal kernel decays at rate 0.55. It does not. The best geometric kernel in the r_1 -family decays at rate

$$\rho^* = 0.464 \quad (\text{measured ratios } 0.462, 0.464, 0.465, 0.466),$$

faster than the process (CE = 1.089316 vs ζ -kernel 1.089562; and letting the kernel vary freely per destination — 55 parameters instead of 10 — buys under **3 micronats** more, $1.089316 \rightarrow 1.089313$). Why does optimal decay undershoot the process eigenvalue?

Worked example (2 correlated regressors; illustrative arithmetic). The constrained update weights lag τ by the *marginal* coupling ζ^τ — the covariance a single lagged token has with the target if it were the only evidence. But consecutive tokens are correlated (that is what ζ *is*), so their evidence overlaps, and a sum of marginal weights counts the overlap twice. Strip it to two predictors a, b of a target y , each with unit target-covariance $\mathbb{E}[ay] = \mathbb{E}[by] = 1$ and unit variance $\mathbb{E}[a^2] = \mathbb{E}[b^2] = 1$, but mutually correlated $\mathbb{E}[ab] = \zeta$. Used alone, each wants coefficient 1. Used *together*, the normal equations $\begin{pmatrix} 1 & \zeta \\ \zeta & 1 \end{pmatrix} \begin{pmatrix} \beta_a \\ \beta_b \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ give

$$\beta_a = \beta_b = \frac{1}{1 + \zeta} = \frac{1}{1.55} = 0.645,$$

each shrunk to 64.5% of its marginal value so the two do not double-count their shared part. Add a *third* correlated regressor and the shrinkage compounds, pushing the effective lag-to-lag ratio below ζ ; the exact constrained-belief version of this partial-regression shrinkage lands the optimal rate at 0.464. And once the marginal-weight kernel over-counts, *adding* another over-counted tail term makes CE worse — the negative rungs of the ladder.

The shrinkage is not a CE artifact. Scanning the matched geometric family under *squared error* instead of cross-entropy gives an MSE-optimal rate of 0.467 (`close_gaps.json`) — essentially the same 0.46, not $\zeta = 0.55$. CE and MSE agree on the shrunk rate: it is a property of the correlated evidence, a P3 Toeplitz-optimality echo, not of the particular loss.

2.4 The trap: the row the gradient never touched

Now the mistake this section exists to prevent. Config A has context length 10; the query at position t (0-based) predicts token $t+1$. The last *scored* query is position 8 (it predicts token 9). **Position 9 would predict token 10, which does not exist** — so no loss term ever references destination row 9, and *no gradient ever flows through it*. Its attention pattern is initialization noise, frozen.

Worked consequence. If you fit “the decay rate of the trained attention kernel” by pooling all destination rows — including row 9 — you read a rate of 0.68–0.71, comfortably *above* $\zeta = 0.55$, and you would conclude the model *contradicts* the spectral ansatz. Restrict to the rows the loss actually trained (rows 1–8) and the fitted rate is 0.43–0.45, matching the CE-optimal 0.464 and cleanly *below* ζ (Fig. 1, right: the gray gradient-free row sits an order of magnitude off, with a spurious bump at lag 4). The pooled fit was dominated by one row of noise.

Moral. Before reading structure off a trained model, find out what the loss *pinned*. In this world that is: lag 0, most of lag 1, a sliver of lag 2, and rows 1–8. Everything else — the geometric tail, row 9 — is loss-flat, and any FRA attribution placed there is reading seed noise dressed as mechanism. This one fact ($\tau \geq 3$ and row 9 are free) will explain, in Section 3, why the FRA-QK “removal” effect can be dialed across a $140\times$ range without the loss noticing.

What would fool you. The two decoys here are opposite in sign. The loss-flat *tail* tempts you to interpret wiggles that cost microNats to erase; the untrained *row* tempts you to interpret a rate that is pure initialization. Both look like “the model does something interesting at long range.” Neither survives asking the loss.

3 The dial: what FRA-QK actually measures

3.1 The question

FRA-QK decomposes each attention *score* into feature×feature pieces and asks which token-pair terms drive who-attends-to-whom. In this world the token-pair terms carry a real $\approx 10\%$ of the score mass. So: cut them, and how much does the model’s behavior change?

3.2 The four score blocks

Write the residual at a position as token feature plus positional feature, $x_s = e(z_s) + p_s$, and split query and key the same way, $q_d = q_E(z_d) + q_P(d)$, $k_s = k_E(z_s) + k_P(s)$. The score is bilinear, so it splits into *four* blocks (formal note Eq. (S2.2)):

$$\sigma(d, s) = \underbrace{q_P(d) \cdot k_P(s)}_{\text{pos} \times \text{pos}} + \underbrace{q_P(d) \cdot k_E(z_s)}_{\text{pos} \times \text{tok}} + \underbrace{q_E(z_d) \cdot k_P(s)}_{\text{tok} \times \text{pos}} + \underbrace{q_E(z_d) \cdot k_E(z_s)}_{\text{tok} \times \text{tok} = Q(z_d, z_s)}.$$

FRA-QK’s “feature×feature score mass” is the last block Q . In our Config A checkpoint it is about 10% of the pattern-weighted score budget (`gauge_qk.json`, `tok_tok_share` ≈ 0.10 across the whole dial below). Ten percent is not nothing, and the naive attribution logic says: cut it and the routing should move.

3.3 One softmax row, and why a cut is a tilt

The reason it will *not* move is one identity. A row of attention is a probability distribution $A_s = e^{\sigma_s} / \sum_{s'} e^{\sigma_{s'}}$ over keys, and the head output is the expectation $c_d = \sum_s A_s v_s = \mathbb{E}_A[v]$. Any score edit is $\sigma \rightarrow \sigma - c \delta$, and its first-order effect on the output is *a covariance across the row*:

$$\Delta c_d = -c \text{Cov}_A(\delta, v) + O(c^2 \text{osc}(\delta)^2).$$

So an edit moves the output only insofar as δ *covaries with the values* v down the row. Two consequences drive everything below: (a) an edit that is *constant across keys* does exactly nothing (softmax kills row constants); (b) the magnitude of a token cut is set by whatever scalar multiplies the indicator $\mathbb{I}[z_s = z^*]$ in δ . For a key-side cut of token z^* that scalar is the same for every token — the **pedestal** $\beta(d) = q_d \cdot \bar{k}_E$ (formal note Eq. (S2.5)). Cut effect $\propto \beta$. Keep that.

3.4 Worked example: dialing the pedestal to zero

Here is the whole point of FRA-QK in one experiment. There is a *loss-preserving reparameterization* — gauge move G1 — that moves token content into positions and back: $e(z) \rightarrow e(z) + t u$ with every position absorbing $-t u$. Because $x_s = e(z_s) + p_s$ is unchanged, **every residual, activation, and logit is bit-identical** along the entire dial: we measured the max logit difference at $\sim 7 \times 10^{-7}$ (float32 roundoff) for every $t \in [-1, 2]$ (`gauge_qk.json`, `identity_max_logit_diff`). The model is, forward-pass, the same model at every t .

But G1 *does* move where the split puts “token content,” so it moves the pedestal β , and therefore the FRA-QK cut coefficient. Sweeping t and recording (i) the per-token pedestal and (ii) the exact-enumeration ΔCE of the operation “cut token 0’s key-side term at gain 1”:

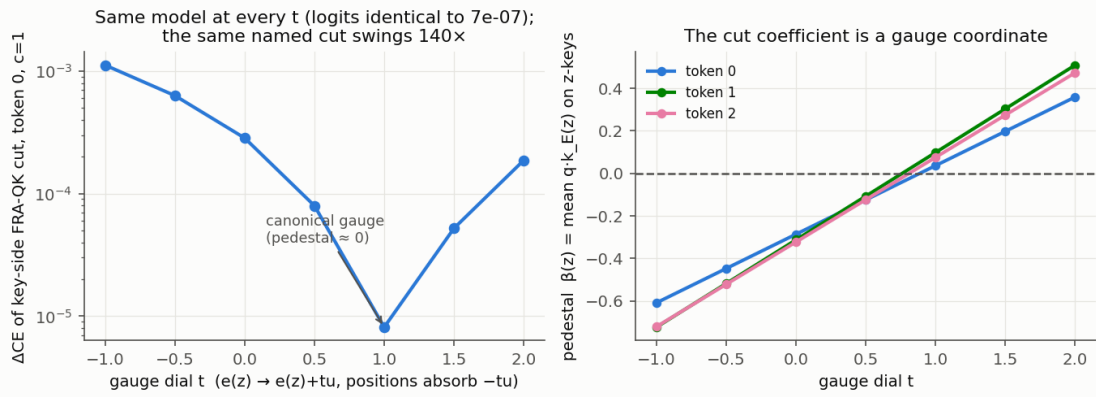


Figure 2: **The G1 gauge dial.** As the embedding/position re-split parameter t sweeps, every logit is unchanged (max difference $\sim 7 \times 10^{-7}$), yet the *named* operation “cut token 0’s key-side FRA-QK term at gain 1” swings its ΔCE over 140 $\times$, minimized exactly where the measured pedestal β crosses zero.

dial t	-1.0	0.0	1.0	1.5	2.0
pedestal $\bar{\beta}$ (mean)	-0.69	-0.31	+0.07	+0.26	+0.45
ΔCE (nats)	1.1×10^{-3}	2.8×10^{-4}	8.1×10^{-6}	5.2×10^{-5}	1.9×10^{-4}
identity logit diff	6×10^{-7}	0	6×10^{-7}	7×10^{-7}	6×10^{-7}

At $t = 0$ the pedestal per token is $-0.287, -0.313, -0.323$; it crosses zero at $t \approx 1$ (there $+0.036, +0.097, +0.074$); the ΔCE of the *same named cut* runs 1.1×10^{-3} to 8.1×10^{-6} to 1.9×10^{-4} , a **V-shape of range 138 $\times$** whose floor sits where $\beta \approx 0$. The behavior of the model never changed; only the coordinate did.

The floor itself is telling: 8.1×10^{-6} nats is *below* the loss-flat scale of Section 2. The gauge-invariant content of the FRA-QK cut is indistinguishable from the noise the loss never sees. (Why so inert at heart? The target kernel $\zeta^{d-s}g(z_s)$ is *separable* — a lag scalar times a content vector — so the token-pair score block carries no causal handle at any minimizer; formal note Thm. boundary. Mess3 is on the null side of that boundary.)

3.5 A corroborating null: cut every token and nothing should happen

The theory says cutting the *whole* token set on the key side is *exactly inert*: every key carries one token, so the summed edit is a per-row constant $\beta(d)$, and constants die in softmax. On the idealized model this holds to 2×10^{-16} . On the *trained* model it does not quite: the full-token-set cut costs $\Delta\text{CE} = 3.0 \times 10^{-4}$ at $t = 0$, about the same as a single-token cut (2.8×10^{-4}). The reason is *content leak*: the trained pattern is not perfectly token-independent (its premise-violation ε is $1.5 \times$ the positional profile slope, `gauge_qk.json`), so the summed edit is not perfectly constant. The theory predicts exactly this: an $O(c\varepsilon)$ residue, pure leak. The “exactly inert” law is an $\varepsilon=0$ statement; on a real checkpoint you see the leak, not zero.

Moral. A number that a loss-preserving reparameterization can send to zero is a *gauge coordinate*, not a mechanism. The FRA-QK “token-0 removal effect” is such a number: same model, same enumeration, 140 $\times$ range at bit-identical behavior. Before you report an attention-score attribution, *dial the gauge* and see whether your effect survives.

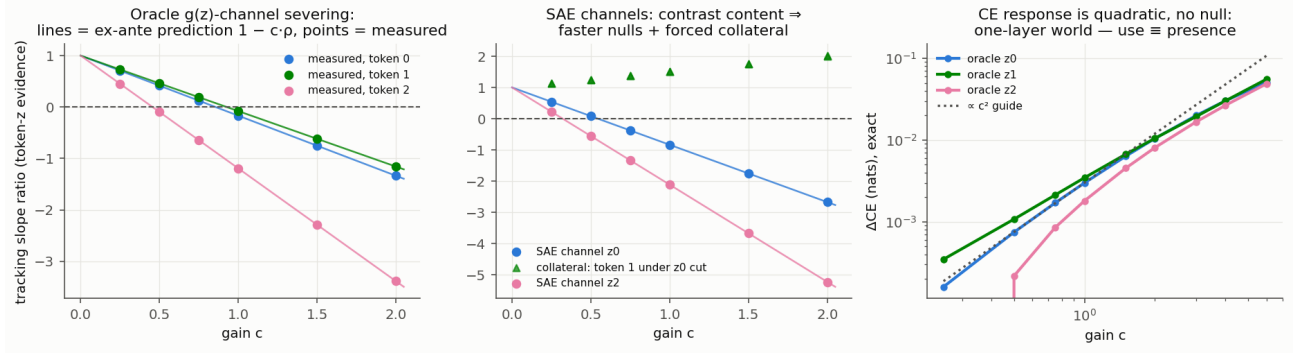


Figure 3: **FRA-OV is causal.** Left: severing a token’s channel drops its belief-tracking *linearly* in the gain c , crossing zero at the predicted null $c^* = 1/\rho_{\text{eff}}$. Middle: geometrically forced collateral — severing token 0 *raises* the other tokens’ tracking. Right: CE removal is *quadratic* in c with no behavioral null (slope-2 guide), because a one-layer model has no redundant path.

What would fool you. The 10% “feature×feature score mass” is real and it is a trap. Score mass is not causal mass. Here the entire token-dependent part of the score block is additively separable into gauge coordinates (formal note Eq. (S2.3)), so it can be made to look like 0% or 100% of the mechanism without touching a single logit. The only invariant — the centered interaction $\hat{q}_E(z) \cdot \hat{k}_E(z')$ — is identically zero in this world.

4 Cutting what flows: FRA-OV is the real handle

4.1 The question

FRA-QK was gauge because it edits *scores*, and scores are one reparameterization away from anything. FRA-OV edits the attention *output*: it subtracts the content a token’s channel actually transports into the residual stream. That content is causal. The questions become quantitative: if we sever token z^* ’s channel at gain c , how far does the model’s belief-tracking fall — and can we predict the number *before running a single edited forward pass*?

4.2 The attribution, and why it over-carries: the warp

The per-(token, lag) content the head transports is (formal note Eq. (S3.1)) the constrained-update term itself, $\Delta_{s \rightarrow d} = A_{d,s} f(v_s) = \zeta^{d-s} g(z_s)$. But there is a subtlety that decides whether ex-ante predictions land: with additive positional embeddings, the softmax-realizable head output is not u_d but a *token-independent multiplicative warp* of it (formal note Eq. (S1.7); twin’s Finding 5),

$$f(c_d) = \lambda_d u_d, \quad \lambda_d = \frac{1}{1 - \zeta^d} \geq 1.$$

Worked, for $\zeta = 0.55$:

$$\lambda_{1..5} = \left(\frac{1}{1-0.55}, \frac{1}{1-0.55^2}, \dots \right) = (2.22, 1.43, 1.20, 1.10, 1.05) \rightarrow 1.$$

The head *over-carries* at small d (a normalization overshoot $\zeta^d/(1 - \zeta^d)$, largest early), then relaxes to unity. This warp is one of the two reasons the tracking slope ρ_{eff} will exceed 1 below; the other is the CE-shrinkage of Section 2 (the model’s readout, unlike the clipped r_1 , scales content up).

4.3 The skip/diagonal split (the $k=0$ gauge)

One subtlety before we cut. The current token’s own displacement $g(z_d)$ (lag 0) reaches the readout by *two* routes: the residual **skip** connection, and **diagonal attention** (position d attending to itself). Only their sum is pinned; the split is a gauge (the $k=0$ degree of freedom). An OV cut removes only the diagonal share. Measured on the trained model, the lag-0 $g(z)$ -projection splits as (`refine_rho.json`):

token	diagonal share	skip share	diagonal fraction
z_0	0.824	0.142	0.85
z_1	0.801	0.148	0.84
z_2	0.561	0.249	0.69

So $\approx 80\%$ of tokens 0 and 1’s lag-0 content is on the (cuttable) diagonal, but only $\approx 69\%$ of token 2’s. That asymmetry will show up again in token 2’s numbers. (The rows do not sum to exactly 1 — 0.97, 0.95, 0.81 — because of the warp and shrinkage above; the shortfall is real, not rounding.)

4.4 Worked example: predicting the tracking slope ex ante

Now the headline. Define the **tracking slope** ρ_{eff} : as we sever token z^* ’s channel at gain c , the model’s belief coupling to z^* -evidence falls as $1 - c\rho_{\text{eff}}$. We predict ρ_{eff} from *clean-model statistics only* — no edited forward passes — by a covariance recipe (`refine_rho.py`):

$$\rho_{\text{eff}} = \frac{J}{E^2 \cdot \text{slope}_0}, \quad J = \left\langle \sum_{s \leq d: z_s = z^*} A_{d,s} \text{proj}_{g(z^*)}(f(v_s)) , \omega_{z^*}(d) \right\rangle_P,$$

where J is the (probability-weighted, over all d) covariance of the removed channel’s $g(z^*)$ -projected content with the true evidence $\omega_{z^*}(d) = \sum_{s \leq d: z_s = z^*} \zeta^{d-s}$; $E^2 = \langle \omega_{z^*}^2 \rangle_P$ is the evidence variance; and slope_0 is the clean model’s belief-tracking slope, which converts the answer into clean-model units.

Do token 0, oracle basis, by the numbers (`refine_rho.json` fields `rho_absolute_units_J_over_E2` and `rho_eff_predicted`; `ov_edits.json`):

$$\frac{J}{E^2} = 0.8766, \quad \text{slope}_0 = 0.7503, \quad \rho_{\text{eff}} = \frac{0.8766}{0.7503} = \mathbf{1.1683}.$$

The measured slope, from actually running the edited model over all 3^{10} sequences, is **1.1683**. Not close — equal to the digits shown. And it is not a one-off; the same recipe nails all five channels we edited:

channel	J/E^2	slope_0	predicted ρ_{eff}	measured
oracle z_0	0.8766	0.7503	1.1683	1.1683
oracle z_1	0.9138	0.8470	1.0789	1.0789
oracle z_2	0.8114	0.3699	2.1935	2.1935
SAE z_0	1.3794	0.7503	1.8384	1.8384
SAE z_2	1.1534	0.3699	3.1181	3.1181

Every row agrees to the four significant figures printed (in fact to $\sim 10^{-7}$; we re-derived the table independently). This is FRA-OV working exactly as an attribution should: the clean model’s own statistics tell you, in advance, how much a channel carries.

4.5 The null, for free

Because tracking falls as $1 - c \rho_{\text{eff}}$, the *concept-nulling gain* — the c at which token z^* ’s tracking hits zero — is predicted before any edit:

$$c^* = \frac{1}{\rho_{\text{eff}}}.$$

For oracle z_0 : $c^* = 1/1.1683 = \mathbf{0.856}$. Sweep the edited model and the tracking ratio crosses zero at $c \approx 0.856$ (it is $+0.124$ at $c = 0.75$, -0.168 at $c = 1.0$). Predicted before measured, to three figures. Note $c^* < 1$: *exact* severing ($c = 1$) here *over*-reaches, because the diagonal share is large (Section above). The folk intuition “severing always under-reaches” is simply false in this world.

4.6 Collateral is geometrically forced

Look at the middle panel of Fig. 3. Sever token 0 and the *other* tokens’ tracking *rises*: at $c = 1$, token 1’s tracking ratio has climbed to $1.17\times$ and token 2’s to $1.39\times$ while token 0’s has been driven to -0.17 . This is not a leak or a method flaw. It is forced by the geometry of Section 1: the three displacements $g(0), g(1), g(2)$ live in a 2-dimensional plane and sum to zero, so *no two are orthogonal* and no single direction is orthogonal to the other two. Pushing the belief’s $g(0)$ -component down necessarily moves its $g(1)$ - and $g(2)$ -components (each $g(z)$ has a nonzero $g(0)$ -projection because $g(1) + g(2) = -g(0)$). **“Selective concept removal” is impossible here — for dimension-counting reasons, before any question of SAE quality or edit precision.** You need to cut ≥ 2 of the 3 tokens to collapse the whole plane; one cut only flattens one direction (the reachable set drops from isotropic to a near-1-D curve, one plane eigenvalue $0.097 \rightarrow 0.016$, formal note §3.4).

4.7 Quadratic CE, and the missing null

The tracking null ($c^* = 0.856$) is *not* a CE null. The right panel of Fig. 3 shows ΔCE rising *quadratically* in c (slope-2 on log axes), with its minimum at $c \approx 0$ — because for $x = 0.15$ the constrained predictor is essentially Bayes-optimal, so the CE-minimizing amount of cut is “don’t cut.” There is **no gain-tuned behavioral null** to find in this world. That matters because the previous sprint’s headline result — a gain-tuned edit that produces a full behavioral null while a retrained probe still reads the concept at $R^2 = 0.985$ — lived on the redundancy between *two* computational paths. A one-layer model has only one path, so “use” and “presence” collapse into the same quantity: severing the use *is* severing the presence. “Presence without use” requires multi-path aggregation; it cannot exist in P2’s architecture.

4.8 The honest misses

Three numbers did not behave, and pretending otherwise would defeat the point of an exactly-solvable world.

(1) Window cuts overshoot the ladder, and the tail is not free. The value ladder (Section 2) says lags $\tau \geq 3$ are worth $\leq 2 \times 10^{-5}$ nats, so cutting that window should be nearly free. It is not, and there are two distinct effects to separate. First, a *full-content* window cut removes the window’s *entire* content — including the normalization-pedestal components the MLP relies on, not just its $g()$ -content — so cutting all $\tau \geq 3$ transport at $c = 1$ costs $\Delta\text{CE} = 2.7 \times 10^{-4}$, an order of magnitude over the ladder; that is built-in collateral. Isolating just the $g()$ -content (now run, `close_gaps.json`) removes most of it: the $g()$ -content-only cuts cost $1.7 \times 10^{-3} / 3.1 \times 10^{-4} / 6.6 \times 10^{-5}$ at $\tau = 1 / \tau = 2$

$/ \tau \geq 3$, versus the ladder’s $1.1 \times 10^{-3} / 1.1 \times 10^{-4} / -4.2 \times 10^{-5}$. Rank order holds over 1.5 decades and the excess is only $1.5\text{--}3\times$ (about the warp scale) — but note the *sign* at the tail: the ladder predicts removing $\tau \geq 3$ should *help* (-4.2×10^{-5}), and the measured cut instead *hurts* ($+6.6 \times 10^{-5}$). The trained model genuinely uses its tail content, at the ~ 7 -microNat loss-flat level the ladder called free. Even the honest, $g()$ -only prediction slightly misses — because “loss-flat” meant “the loss barely cares,” not “the model does nothing there.”

(2) Token 2’s clean tracking slope is anomalously low (0.370). The clean tracking slopes are (0.750, 0.847, 0.370) for z_0, z_1, z_2 . Token 2’s is half the others’, an unexplained trained-readout asymmetry (it partly tracks the lower diagonal share, 0.69 vs 0.85, but not fully). Because $\rho_{\text{eff}} = J/(E^2 \cdot \text{slope}_0)$ divides by this slope, token 2’s channels get inflated ρ and c^* (2.19 and 3.12). The *prediction* still lands — the anomaly is in both the recipe and the measurement, so it cancels — but the absolute number is odd and we cannot account for the factor of two.

(3) The content leak is not small. The premise that the pattern is token-independent is violated at $1.5\times$ the profile-slope scale (Section 3). The OV predictions survive because they use the *measured* clean pattern, not the idealized one; but every “ $\varepsilon = 0$ ” identity (like the exactly-inert full-token cut) is only an approximation here.

Moral. FRA-OV is the real handle: linear response, a slope predictable to four figures from the clean model, a null you know before you cut. But “predictable” is not “surgical.” The same geometry that makes the prediction exact (2 dimensions, 3 summing-to-zero directions) makes selective removal impossible, and full-content window cuts drag along non- $g()$ content the ladder never priced. Attribution you can trust to the fourth figure, and collateral you cannot avoid, are the *same* fact about this world.

5 The dictionary you deserve

5.1 The question

Sections 3–4 used an *oracle* basis — we knew $g(z)$ analytically. In practice you get your basis from a sparse autoencoder. So: when you learn the code instead of knowing it, what do the “token channels” you cut actually contain? This is the cleanest habitat FRA will ever see (the SAE input is a deterministic function of (token, position) with only 30 distinct values), so whatever goes wrong here is a floor on what goes wrong everywhere.

5.2 Worked example: what TopK learned

A TopK SAE (12 latents, $K = 3$) on `resid_pre` reconstructs to $\text{FVU} = 2.3 \times 10^{-6}$ — essentially perfect — and (all three SAE seeds agree) it partitions its latents as

9 positional + 2 token + 1 dead.

Two token latents, for three tokens. The token-to-latent map is $\{0 \mapsto \{11\}\}, \{1 \mapsto \{\}\}, \{2 \mapsto \{5\}\}$: **token 1 has no latent of its own.** It is encoded by absence — when neither token latent fires, the decoder bias supplies token 1. So “sever token 1’s channel” is not an operation that exists in this latent set. (In the oracle basis it exists and works: $\Delta\text{CE} = 3.5 \times 10^{-3}$.) The obstruction is the code’s minimality, not the concept’s reachability — a concrete instance of the rank-deficiency of single-feature cuts.

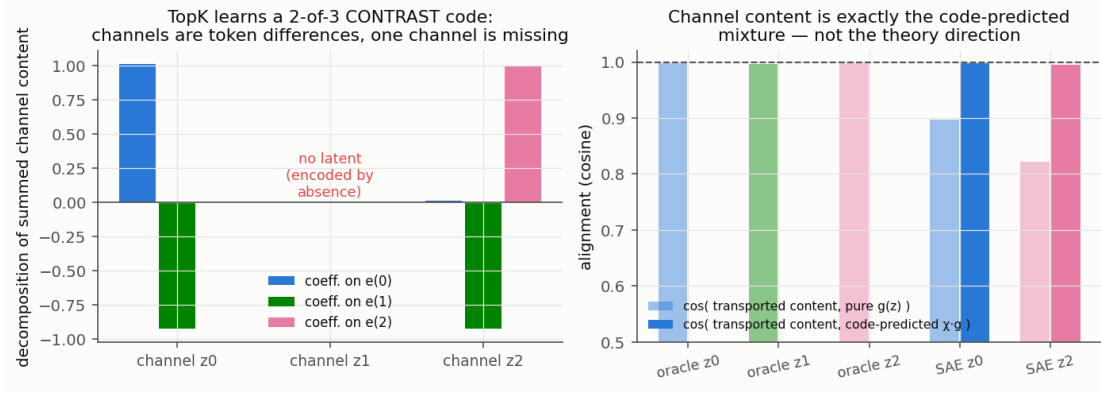


Figure 4: **The learned code is a minimal contrast code.** A TopK SAE on `resid_pre` learns 9 positional latents and 2 token latents for 3 tokens; token 1 is encoded by *absence*. The two token channels decode as *contrasts* against the absent token ($u_0 \approx 1.01 e(0) - 0.93 e(1)$ and $u_2 \approx 1.00 e(2) - 0.93 e(1)$), not pure theory directions.

5.3 The channels are contrasts, not directions

Decode the summed token-0 channel back onto the token embeddings (least-squares of its mean content on $[e(0), e(1), e(2), 1]$ over the channel’s support; `dict_deep.py`). Its token-content row is

$$\chi_0 = (1.012, -0.927, -0.000) \implies u_0 \approx 1.01 e(0) - 0.93 e(1),$$

and token 2’s is $\chi_2 = (0.012, -0.926, 1.000)$, i.e. $1.00 e(2) - 0.93 e(1)$. Both channels carry a $-0.93 e(1)$ term: token 1, the absent one, is the shared reference both contrasts subtract. So the channels do not transport pure $g(z)$; they transport a *mixture* $\chi_z \cdot g()$. Check it against the actual transported content (`dict_deep.json`, verified with both an independently-fit belief projector):

cosine of transported channel content with	token 0	token 2
the code-predicted mixture $\chi_z \cdot g()$	0.9992	0.9953
the pure theory direction $g(z)$	0.90	0.82

The transported content matches the *mixture* to cosine 0.999, but is 26° off (cosine 0.90) from the pure direction $g(0)$: about 27% of the channel’s content (χ -residual fraction 0.27) is off-theory. If you assumed “SAE token-0 channel = $g(0)$ ” you would edit along the wrong axis; the code hands you a contrast, and you must extract the mixing matrix χ before your FRA edit means what you think. (*Honesty note, worth internalizing*: an earlier version of this script estimated χ from an aliased cell-design regression and stored a mixture cosine of 0.07 — badly wrong. The regenerated `dict_deep.json` fits χ cleanly over the channel’s support and reports 0.9992. The lesson is exactly the section’s: *how you fit the code decides what the edit means*.)

5.4 The intervention inherits the code

This is not academic. Run the FRA-OV cut of token 0 *through the SAE channel* instead of the oracle direction. Because the channel is the contrast $1.01g(0) - 0.93g(1)$, cutting it does two things at once (`ov_edits.json`): it nulls token 0’s tracking at $c \approx 0.54$ (predicted $c^* = 1/1.838$, Section 4.4), *and* it *boosts* token 1’s tracking to $1.5\times$ at $c = 1$ — because the channel’s $-0.93 e(1)$ component injects

negative token-1 content, which reads as more token-1 evidence once removed. The oracle cut and the SAE cut null the same token at different gains and collateralize different neighbors: the edit is exactly as clean as the code, and no cleaner.

5.5 Purity does not see this

Here is the sharp part. By the previous sprint’s criterion — set-level token *purity* — these channels are perfect: $R^2 \approx 1.0000$, each token latent fires on exactly its token. Purity is real and it is *not sufficient*. A channel can be perfectly pure and still be a contrast ($1.01g(0) - 0.93g(1)$), still be missing entirely (token 1), still transport a mixture. **What FRA edits actually inherit is the code’s completeness and centering**, and purity is blind to both. The one diagnostic that would have caught it — does the channel exist, and does its transported content point where theory says — is the mixing-matrix check above, not the purity R^2 .

One reassuring measurement to close the section: the deeper `resid_mid` SAE does *not* hide the computation in its error term. Its reconstruction error carries only 3.3% of the belief-plane variance while being 5.9% of the total residual variance (`dict_deep.json`): the error is *less* belief-laden than its size, so the SAE is not sweeping the belief signal into the residual. When completeness and centering are good, FRA is faithful; the failure mode is the code, not the method.

What would fool you. A clean purity table ($R^2 = 1.0$) reads as “the SAE found the concept.” It found a *pure* code, which is necessary but says nothing about whether the code is complete (token 1 is missing) or centered on theory directions (the channels are contrasts). Trusting purity, you would cut “token 0” and silently also inject token-1 content — and your purity table would still say 1.0.

6 Two heads, one truth

6.1 The question

Everything so far was Config A, $\zeta = +0.55$, one head. Flip to **Config B**: $x = 0.50$, so $\zeta = 1 - 3x = -0.50$. Now the process *anti*-correlates: even lags help, odd lags hurt. What does a single softmax head do with a sign that flips every step — and what happens to per-head FRA attributions when the answer is “it can’t”?

6.2 Why one head cannot: a one-line argument

The target per-source coefficient is $\zeta^{d-s} = (-1)^{d-s}|\zeta|^{d-s}$ — it alternates sign with lag. But a softmax attention row is a probability distribution: *every weight is non-negative*. A non-negative pattern times a single OV direction cannot produce a sign that flips. So P2’s solution is forced: **two heads with anti-parallel OVs**. Head 1 (OV along $+g()$) carries the even lags; head 2 (OV along $-g()$) carries the odd lags; the head-*sum* reproduces the oscillation,

$$\sum_{h=1,2} A_{d,s}^{(h)} f(v_s^{(h)}) = (-1)^{d-s} |\zeta|^{d-s} g(z_s) = \zeta^{d-s} g(z_s).$$

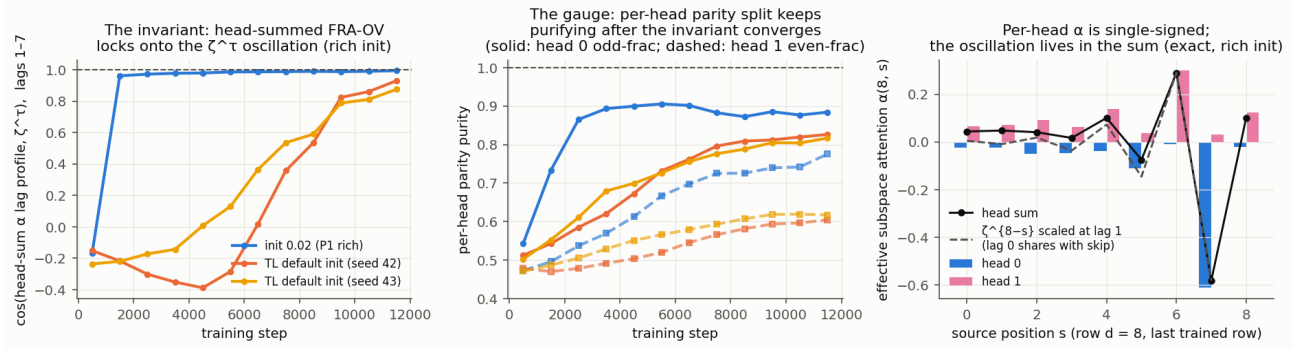


Figure 5: **The two-head hinge at $\zeta = -0.5$.** The head-summed effective attention locks onto the signed ζ^τ oscillation as training proceeds (cosine climbing), while the per-head parity split keeps drifting at nearly constant loss. Rich (small) init converges the invariant faster than TransformerLens-default init.

6.3 Worked example: the oscillation lives only in the sum

Take the last trained row ($d = 8$) of our cleanest Config B checkpoint and read off the per-head effective attention (`twohead.json`; formal note H1):

head 0 coefficients $\alpha(8, \cdot)$	<i>all negative</i> (OV anti-parallel to $g()$)
head 1 coefficients $\alpha(8, \cdot)$	<i>all positive</i> (OV parallel to $g()$)
head-sum $\alpha(8, \cdot)$	oscillates: $\dots, +0.29, -0.58, +0.10$
theory ζ^{8-s}	$\dots, +0.25, -0.50, +1.00$
$\cos(\text{head-sum}, \zeta^\tau)$, row-8 clean per-cell	0.985

Neither head alone can carry the alternation — each is single-signed. The signed ζ^{d-s} kernel appears *only* in the sum, and there (measured cell-by-cell on the clean row-8 pattern) it is recovered at cosine 0.985. **The head-sum is the invariant; the per-head split is not.**

6.4 The invariant converges; the split never stops moving

Track training (checkpoints every 500 steps, `twohead.json`). The head-sum’s alignment with ζ^τ climbs and settles. On the lag-1–7 profile (the pinned tail, excluding the lag-0 term that the skip/diagonal split depresses) our rich-init run goes from cosine -0.17 at step 500 to **0.994** by the end — essentially exact. (The lag-0-included estimator plateaus lower, at 0.72, precisely because lag-0 content is shared with the skip path; and the row-8 per-cell measurement above reads 0.985. Same invariant, three estimators.) Meanwhile the *per-head parity shares keep drifting* after the loss and the invariant have both converged: head-1’s even-lag fraction is still moving over the final thousands of steps at essentially constant CE. The decomposition is a loss-flat gauge coordinate; the sum is pinned.

6.5 Rich vs. lazy init decides even the invariant

One more wrinkle, the P2 App-G rich/lazy effect reproduced here (numbers on the lag-0-included estimator, which resolves the init differences most sharply). With small (“rich”) init the head-sum invariant converges smoothly ($0.23 \rightarrow 0.72$). With TransformerLens-default (“lazy”) init one seed’s

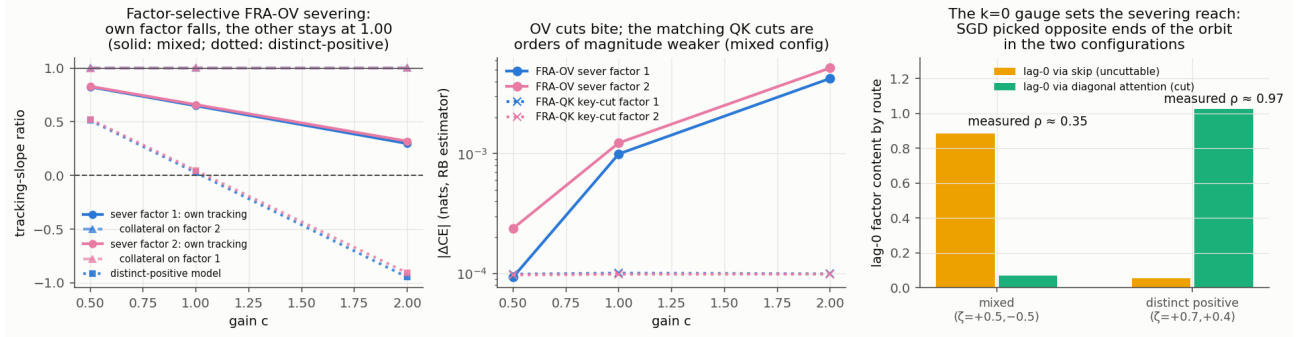


Figure 6: **Same cut, two runs, opposite reach — and perfect cross-factor protection.** Severing factor 1’s OV channel severs $\rho \approx 0.97$ per unit gain in the distinct-positive model but only $\rho \approx 0.35$ in the mixed model (left) — an inter-model instance of Phase 1’s $c^* = 1/\rho$ gauge law. Cross-factor tracking (right) stays at $+1.00$ at every gain in every model: orthogonal subspaces mean no forced collateral, unlike Mess3’s $1.5\times$.

head-sum cosine *dips* to 0.07 mid-training before recovering to 0.63 — still climbing at the end of P2’s training budget — while another default-init seed reaches 0.75. So even the *invariant*’s convergence, not just the gauge split, is init-dependent within a fixed budget. All seeds do pick the same canonical even/odd parity assignment; they just get the sum sharp at different rates.

Moral. For $\zeta < 0$, per-head FRA-OV rankings are reads of a *gauge orbit* at a particular checkpoint and a particular init — they drift at constant loss and depend on initialization. Only the head-sum $\sum_h A^{(h)} f(v^{(h)}) = \zeta^{d-s} g(z_s)$ is a trustworthy attribution. **Sum the heads before you interpret.** A per-head bar chart is a snapshot of a moving coordinate; the sum is the physics.

What would fool you. A per-head attribution table at $\zeta < 0$ looks like a finding: “head 1 does even lags, head 2 does odd.” It is true *at this checkpoint*, and it keeps shifting while the loss sits still — and a key-side cut applied at one gain hits the two heads through *different* normalizers, so it perturbs the head-sum in a way that does not cancel. Read per-head numbers as gauge, never as mechanism.

7 Coda: the factored world (Phase 2)

Phase 2 moves to P1’s **factored** process: two Mess3 components running in parallel, $T = T^{(1)} \otimes T^{(2)}$, with their own eigenvalues ζ_1, ζ_2 in two *approximately orthogonal* 2-dimensional residual subspaces (three P1-faithful models, 100k steps, all within 0.17–0.20 millinats of Bayes). The whole apparatus of this note lifts per subspace: each factor has its own update $\Delta r_n = \sum_s \zeta_n^{d-s} g_n(z_s^{(n)})$, and the FRA-OV attribution applies inside V_{F_n} . Two results are worth the coda.

Both eigenvalues from one model, and a dead head you can see in one number. The per-factor effective subspace attention recovers each factor’s ζ_n from a *single* trained model: distinct-positive ($\zeta = (0.7, 0.4)$) gives lag-ratios 0.705 / 0.381 (cosine with ζ_n^T 0.997 / 0.9995); mixed ($\zeta = (0.5, -0.5)$) gives 0.572 / -0.570 vs ± 0.5 , the sign oscillation living only in the head-sum as in Section 6. How heads divide the labor is itself a finding: the distinct-positive model *specializes* (head-factor share matrix $\begin{bmatrix} 0.95 & 0.28 \\ 0.05 & 0.72 \end{bmatrix}$, one head per factor), the mixed model *collaborates* ($\begin{bmatrix} 0.64 & 0.64 \\ 0.36 & 0.36 \end{bmatrix}$, roles split by parity

not factor). And in the three-head mixed model the third head carries share 0.0005 / 0.00006 of the two factors — **a dead head**, P1’s conic minimum-head-count $H_{\min} = 2$ with ray reuse, visible in one number.

Cross-factor collateral is exactly zero — the prediction from Section 4 confirmed. In P2’s single Mess3 selective removal was impossible because the three $g(z)$ share one 2-plane. Here the factors live in orthogonal subspaces, so severing factor n should leave factor m untouched — and it does, to the digit: factor m ’s tracking sits at +1.00 at *every* gain in *every* model (Fig. 6, right), against Mess3’s geometrically forced $1.5\times$. FRA-QK factor cuts stay inert (-1 to -3.4×10^{-4} nats, gain-independent, tracking 1.00); FRA-OV cuts reach 1.5×10^{-2} nats and null tracking — the same QK-gauge / OV-handle split as Phase 1, now per factor.

The teaching-gold: the spectrum, not luck, picks the severing reach. The two headline slopes — distinct-positive severs $\rho \approx 0.97$ (null at $c \approx 1.03$), mixed severs only $\rho \approx 0.35$ — come from the *same* OV cut on the *same* task family. What differs is not the embedding (the token $\times$ position interaction share is 1.1% in both; we tested and refuted the “one model factored its embeddings better” hypothesis first). It is the $k=0$ gauge of Section 4, now varying *across models*: the distinct-positive run delivers its lag-0 factor content through *diagonal attention* (the α lag-0 coefficient is 0.96), so an OV cut catches it; the mixed run delivers the same content through the *skip* (α lag-0 coefficient 0.05), so an OV cut misses it — and at $|\zeta| = 0.5$ that lag-0 term is about half the evidence. And which end of the orbit a model lands on is *not* seed luck: across six models spanning both phases, seeds, and init scales, every positive-spectrum config sits at the diagonal end and every config carrying a negative eigenvalue sits at the skip end (the seed-43 replicas land exactly where their seed-42 twins did). The *data’s spectrum* picks the end of the gauge orbit, not SGD’s coin. Phase 1 warned that “ c^* is a property of the run, not the task” (the $\rho(a_0)$ sweep, Section 4); Phase 2 sharpens it — the run’s gauge position is itself a systematic function of the spectrum. Read a severing gain as a fact about the model’s spectrum and gauge, never about the concept.

And the dictionary lesson recurs. The P1-world SAE learns a complete per-value code for the positive factor (three latents) but gives the *negative* factor a single latent for its three values (11/24 latents mixed, FVU 7.5×10^{-3}) — the completeness gap of Section 5, and it selects against exactly the factor whose content is split across anti-parallel heads. The binding constraint is, again, the code.

What would fool you. You might notice that these Phase-2 rates (0.705/0.381) sit at the *raw* eigenvalues $\zeta = (0.7, 0.4)$, not the shrunk optima of Section 2 — and conclude the shrinkage story was P2-specific. It was not: the per-factor MSE-optimal rates *are* shrunk, to $\eta^* = 0.60/0.34$ (the same 0.85 factor that gave 0.464 in P2), but moving from η^* to raw ζ costs only 2.2% / 0.56% of each factor’s (already microNat) lag-information. The *rate itself* is loss-flat — Section 2’s mechanism applied to the decay parameter rather than the tail values — so SGD sits at raw ζ with no pressure to shrink. (Likewise the product process carries a third eigenvalue $\zeta_1\zeta_2 = 0.28$, the $g_1 \otimes g_2$ cross term of the displacement: its optimal geometric is real but worth $\sim 2 \times 10^{-5}$ nats — present in the theory, invisible to the trained model. Loss-flatness is not one finding; it is the weather of this whole world.)

8 Field guide: five questions before you trust an FRA reading

Everything above compresses to a checklist. Before attributing anything to a feature-resolved attention number, run these five.

- 1. Is the structure loss-pinned?** Compute (or estimate) the CE value of the lag, row, or window you are reading. If the loss never paid for it — the geometric tail, the gradient-free last row (Section 2) — your “mechanism” is seed noise. In this world, only lags 0–2 and rows 1–8 were pinned; everything else was free.
- 2. Is the coefficient a gauge coordinate? Dial it.** FRA-QK cut effects are dialable across $140\times$ at bit-identical behavior by a loss-preserving re-split (Section 3). Before reporting a score attribution, apply the gauge move and watch whether your number survives. If it can be sent to zero, it was a coordinate, not a computation.
- 3. Is the channel complete and centered?** An SAE channel can be perfectly *pure* ($R^2 = 1$) and still be a *contrast* ($1.01g(0) - 0.93g(1)$), or missing entirely (token 1 has no channel; Section 5). Extract the mixing matrix χ and check the transported content points where theory says — purity will not tell you.
- 4. Did you sum the heads?** For $\zeta < 0$, per-head attributions drift at constant loss and depend on init (Section 6). Only the head-sum is the invariant. Sum before you interpret; a per-head bar chart is a snapshot of a moving coordinate.
- 5. Can your edit even be surgical here?** Count dimensions. If the concept directions span fewer dimensions than there are directions (2 vs 3 in Mess3), selective removal is geometrically impossible and *all* of your collateral is forced, independent of method (Section 4). Know this before you blame your SAE.

The through-line: FRA-QK measures gauge, FRA-OV measures a real (but non-surgical) handle, and both are exactly as trustworthy as your answers to questions 1–5. In a 9.4-millinat world where every quantity has a closed form, we could check each answer against the ground truth. On a real model you cannot — which is precisely why the checklist, and not any single number, is the thing to carry over.